

DATA SCIENCE & MACHINE LEARNING

Complete Theory + Practical Python Notes

Purpose: A single study document covering the complete workflow from raw data to a trained, evaluated and deployable machine-learning model.

How to use these notes: Read the theory first, then execute every code block yourself. Do not memorize algorithms only by name. For every algorithm, understand the problem it solves, assumptions, intuition, mathematics, training process, strengths, weaknesses, hyperparameters, evaluation and when to use it.

Recommended environment

```
python -m venv .venv
# Windows
.venv\\Scripts\\activate
pip install numpy pandas matplotlib seaborn scipy scikit-learn jupyter
pip install xgboost lightgbm catboost imbalanced-learn
```

TABLE OF CONTENTS

1. Data Science Overview
2. Python for Data Science
3. NumPy
4. Pandas
5. Data Cleaning
6. Statistics for Data Science
7. Probability
8. Exploratory Data Analysis and Visualization
9. Feature Engineering and Feature Selection
10. Machine Learning Foundations
11. Train / Validation / Test Strategy
12. Data Preprocessing and Pipelines
13. Linear Regression
14. Polynomial Regression
15. Ridge, Lasso and Elastic Net
16. Logistic Regression
17. k-Nearest Neighbors
18. Naive Bayes
19. Decision Trees
20. Random Forest
21. Gradient Boosting
22. XGBoost / LightGBM / CatBoost
23. Support Vector Machines
24. Neural Networks and Deep Learning Foundations
25. Clustering
26. PCA and Dimensionality Reduction
27. Anomaly Detection
28. Association Rule Learning
29. Time Series Foundations
30. Imbalanced Classification
31. Model Evaluation
32. Hyperparameter Tuning
33. Cross-Validation
34. Explainability
35. ML Project Workflow
36. Model Saving and Deployment

37. Common Mistakes and Interview Questions

38. Practical Study Plan

1. Data Science Overview

Data Science is the discipline of extracting useful information, patterns and decisions from data. It combines programming, statistics, mathematics, domain knowledge, data engineering and machine learning.

Data Science vs Machine Learning vs AI

Area	Meaning	Typical work
Data Science	End-to-end use of data for insight and decisions	Collect, clean, analyze, visualize, model, communicate
Machine Learning	Algorithms that learn patterns from data	Train models, tune parameters, predict/classify
Artificial Intelligence	Broader field of systems performing tasks associated with intelligence	Planning, natural language processing, ML, language, agents
Deep Learning	Neural-network based subset of ML	Images, audio, NLP, complex nonlinear patterns

Typical data workflow

```
graph TD
    Problem --> Data_collection[Data collection]
    Data_collection --> Data_understanding[Data understanding]
    Data_understanding --> Data_cleaning[Data cleaning]
    Data_cleaning --> EDA_visualization[EDA / visualization]
    EDA_visualization --> Feature_engineering[Feature engineering]
    Feature_engineering --> Train_validation_test_split[Train / validation / test split]
    Train_validation_test_split --> Preprocessing[Preprocessing]
    Preprocessing --> Model_training[Model training]
    Model_training --> Evaluation[Evaluation]
    Evaluation --> Hyperparameter_tuning[Hyperparameter tuning]
    Hyperparameter_tuning --> Final_model[Final model]
    Final_model --> Deployment_monitoring[Deployment / monitoring]
```

Types of machine learning

Type	Goal	Examples
Supervised	Learn $X \rightarrow y$ from labeled data	Regression, classification
Unsupervised	Find structure without target labels	Clustering, PCA
Semi-supervised	Use a small labeled set plus much unlabeled data	Image/text classification
Self-supervised	Create learning signals from the data itself	Representation learning
Reinforcement	Learn actions from rewards/penalties	Robotics, games

2. Python for Data Science

Core Python topics to know: variables, numeric/string/boolean types, lists, tuples, sets, dictionaries, conditions, loops, functions, comprehensions, exceptions, modules, file handling and virtual environments.

```
# Lists
marks = [80, 90, 75]

# Dictionary
student = {"name": "Asha", "cgpa": 8.7}

# Function
def average(values):
    return sum(values) / len(values)

print(average(marks))

# List comprehension
squares = [x*x for x in range(5)]
```

Useful Python concepts for ML: mutability, shallow vs deep copy, iterables, generators, lambda functions, `\*args/\*\*kwargs`, object-oriented basics, package installation, environments and reading files.

Files you will commonly handle: CSV, Excel, JSON, Parquet, SQL tables and API responses.

3. NumPy

NumPy provides efficient multidimensional arrays and numerical operations. Pandas and many ML libraries build on numerical array concepts.

```
import numpy as np

a = np.array([10, 20, 30, 40])
print(a.shape)      # (4,)
print(a.ndim)       # 1
print(a.dtype)

b = np.array([[1,2,3], [4,5,6]])
print(b.shape)      # (2,3)

print(np.mean(a))
print(np.median(a))
print(np.std(a))
print(np.min(a))
print(np.max(a))
```

Important concepts: indexing, slicing, boolean masking, broadcasting, vectorization, reshaping, transpose, concatenation, aggregation, random generation and linear algebra.

```
x = np.arange(12).reshape(3, 4)

# Boolean filtering
print(x[x > 5])

# Matrix multiplication
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print(A @ B)

# Reproducible randomness
rng = np.random.default_rng(42)
print(rng.integers(0, 10, size=5))
```

Broadcasting: NumPy can apply operations between compatible shapes without explicitly copying values. Understanding shapes prevents many ML bugs.

4. Pandas

Pandas is the main tabular-data tool in Python. A Series is one labeled column; a DataFrame is a table of labeled rows and columns.

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df.head())
print(df.tail())
print(df.shape)
print(df.columns)
print(df.dtypes)
df.info()
print(df.describe())

# Select
cgpa = df["cgpa"]
subset = df[["cgpa", "attendance"]]

# Filter
high = df[df["cgpa"] > 8]

# Multiple conditions
result = df[(df["cgpa"] > 8) & (df["attendance"] > 80)]

# Sort
df.sort_values("cgpa", ascending=False)

# Grouping
df.groupby("branch")["cgpa"].mean()
```

Essential Pandas operations: `read_csv`, `read_excel`, `head`, `tail`, `sample`, `shape`, `info`, `describe`, `loc`, `iloc`, `query`, `sort_values`, `value_counts`, `groupby`, `agg`, `merge`, `join`, `concat`, `pivot_table`, `melt`, `drop`, `rename`, `astype` and `to_csv`.

```
# Missing values
df.isna().sum()

# Duplicates
df.duplicated().sum()

# Unique values
df["branch"].unique()
df["branch"].nunique()

# Save
df.to_csv("cleaned.csv", index=False)
```

5. Data Cleaning

Data cleaning is not a cosmetic step. Incorrect cleaning can change the meaning of the dataset or introduce leakage.

Missing-value strategies

Situation	Possible strategy
Numerical, roughly symmetric	Mean
Numerical, skewed/outliers	Median
Categorical	Mode or explicit Unknown
Very high missingness	Investigate dropping feature/rows
Time-series	Forward/backward fill only when justified

```
# Median imputation example
median = df["cgpa"].median()
df["cgpa"] = df["cgpa"].fillna(median)
```

```
# Categorical mode
df["branch"] = df["branch"].fillna(df["branch"].mode()[0])
```

Outliers: an outlier is an observation unusually far from the rest. It may be an error, a rare valid event, or a signal. Never delete it automatically.

```
Q1 = df["salary"].quantile(0.25)
Q3 = df["salary"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["salary"] < lower) | (df["salary"] > upper)]
```

Data quality checklist: missing values, duplicates, invalid categories, impossible values, inconsistent units, date parsing, incorrect data types, leakage columns, target contamination and suspicious identifiers.

6. Statistics for Data Science

Population vs sample: A population is the complete group of interest; a sample is a subset used to estimate population properties.

Statistic	Formula / meaning
Mean	$\Sigma x / n$
Median	Middle value after sorting
Mode	Most frequent value
Range	max – min
Variance	Average squared deviation from the mean
Standard deviation	$\sqrt{\text{variance}}$; same units as data
IQR	$Q3 - Q1$

Distribution: Understand center, spread, skewness, tails and outliers. Normal distribution is important, but real-world data is often not normal.

Covariance indicates how two variables vary together. **Correlation** standardizes this relationship to approximately –1 to +1 for Pearson correlation.

Correlation is not causation. A high correlation may be due to coincidence, confounding, selection effects or another variable.

Standardization: $z = (x - \mu) / \sigma$. After standardization, a feature has mean approximately 0 and standard deviation approximately 1.

Confidence interval: an interval procedure describing uncertainty in an estimated parameter. Do not interpret it as a probability statement about a fixed parameter.

Hypothesis testing: define H_0 and H_1 , choose a test, calculate a statistic/p-value or confidence interval, then interpret using domain context and effect size.

7. Probability

Probability quantifies uncertainty. ML uses probability in classification, Bayesian methods, uncertainty estimation and statistical reasoning.

Basic rules

$$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$$

$$P(A | B) = P(A \text{ and } B) / P(B)$$

Bayes:

$$P(A | B) = P(B | A) P(A) / P(B)$$

Random variables: discrete variables take countable values; continuous variables take values over intervals.

Expected value: the probability-weighted average of outcomes.

Common distributions: Bernoulli, Binomial, Poisson, Uniform, Normal, Exponential.

Central Limit Theorem: under common conditions, the sampling distribution of the mean tends toward normality as sample size increases, even when the original population is not normal.

Bayes theorem is central to Naive Bayes classifiers and probabilistic reasoning.

8. Exploratory Data Analysis and Visualization

EDA means asking questions of the data before modeling. Good EDA is driven by the problem, not by producing every possible chart.

Question	Useful method
Distribution of one numeric variable	Histogram, KDE, boxplot
Compare numeric distributions	Boxplot, violin plot
Relationship of two numeric variables	Scatter plot
Category counts	Bar chart, value_counts
Category vs numeric	Boxplot/groupby
Many numeric relationships	Correlation matrix/pair plot
Time behavior	Line chart

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df["cgpa"], kde=True)
plt.show()

sns.boxplot(data=df, x="placement", y="cgpa")
plt.show()

sns.scatterplot(data=df, x="cgpa", y="aptitude", hue="placement")
plt.show()

sns.heatmap(df.select_dtypes("number").corr(), annot=True)
plt.show()
```

EDA questions: What is the target distribution? Are classes imbalanced? Which features differ by class? Are there missingness patterns? Are there outliers? Are features redundant? Is there evidence of leakage?

9. Feature Engineering and Feature Selection

Feature engineering transforms raw variables into representations that may make a learning problem easier.

```
# Example
df["skill_score"] = (df["aptitude"] + df["communication"]) / 2

df["experience_score"] = (
    2 * df["internships"] + df["projects"]
)

# Date features
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day_of_week"] = df["date"].dt.dayofweek
```

Encoding: nominal categories commonly use one-hot encoding; ordinal categories can use ordered mappings when the order is meaningful.

Feature selection approaches: domain knowledge, filter methods (correlation, mutual information), wrapper methods, embedded methods (L1 regularization, tree importance) and recursive feature elimination.

Beware leakage: a feature is invalid if it would not be available at prediction time or contains information generated after the outcome.

10. Machine Learning Foundations

A machine-learning model learns a function from examples. In supervised learning, training examples contain inputs X and target y.

```
x = df.drop(columns=["target"])
y = df["target"]

# Conceptually
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Parameters vs hyperparameters

Term	Meaning	Example
Parameter	Learned from training data	Weights in linear regression/neural networks; tree split structure
Hyperparameter	Chosen before/during training by the practitioner or tuning process	k in KNN, tree depth, learning rate, regularization strength

Underfitting: model too simple to capture the pattern. **Overfitting:** model learns training-specific noise and performs poorly on unseen data.

Bias-variance tradeoff: high bias often means underfitting; high variance often means overfitting. Model complexity, regularization and data size influence both.

11. Train / Validation / Test Strategy

Never evaluate a model on the same data used to fit it as your final estimate of generalization.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Train set: fit model and preprocessing parameters. **Validation set:** compare models/tune choices. **Test set:** final unbiased estimate after decisions are complete.

With limited data, cross-validation can replace a fixed validation split for model selection while keeping a separate test set.

Time series: do not randomly shuffle future observations into training. Use chronological splits.

12. Data Preprocessing and Pipelines

Common preprocessing includes missing-value imputation, categorical encoding, scaling, feature selection and sometimes transformations.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_features = ["age", "cgpa", "attendance"]
categorical_features = ["branch"]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features)
])
```

Why Pipeline? It prevents inconsistent preprocessing and helps avoid leakage by fitting transformations inside the training workflow.

13. Linear Regression

Problem solved: Predict a continuous numeric target.

Intuition: Fit a weighted linear combination of input features. The model predicts a plane/hyperplane in feature space.

Core mathematics

$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$. Ordinary least squares minimizes $\sum (y_i - \hat{y}_i)^2$.

Training process

Estimate coefficients that minimize squared residuals. With matrix notation, the least-squares solution is related to $(X^T X)^{-1} X^T y$ when the inverse exists; practical libraries often use numerically stable solvers.

Important hyperparameters

Hyperparameter	What it controls
fit_intercept	Whether to estimate the intercept.
positive	Whether coefficients must be non-negative in supported implementations.

Strengths

- Simple and interpretable.
- Fast to train.
- Strong baseline for approximately linear relationships.

Limitations

- Sensitive to outliers.
- Cannot directly capture complex nonlinear relationships.
- Multicollinearity can make coefficients unstable.

Practical implementation

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

14. Polynomial Regression

Problem solved: Capture nonlinear relationships while using linear regression machinery.

Intuition: Create polynomial features such as x^2 and x^3 , then fit a linear model on those transformed features.

Core mathematics

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d.$$

Training process

Transform X with `PolynomialFeatures`, then fit `LinearRegression` or a regularized model.

Important hyperparameters

Hyperparameter	What it controls
degree	Maximum polynomial degree.
include_bias	Whether to include a constant column.

Strengths

- Can model smooth nonlinear patterns.
- Simple extension of linear regression.

Limitations

- High degrees can overfit.
- Feature count can grow rapidly.
- Sensitive to feature scale.

Practical implementation

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge

model = make_pipeline(
    PolynomialFeatures(degree=2),
    Ridge(alpha=1.0)
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

15. Ridge, Lasso and Elastic Net

Problem solved: Control overfitting in linear models through regularization.

Intuition: Penalize large coefficients so the model prefers simpler solutions.

Core mathematics

Ridge: $\text{loss} + \alpha \sum \beta_j^2$. Lasso: $\text{loss} + \alpha \sum |\beta_j|$. Elastic Net combines L1 and L2 penalties.

Training process

Choose a regularization strength using validation/CV. Scale numeric features because penalty is scale-sensitive.

Important hyperparameters

Hyperparameter	What it controls
alpha	Regularization strength.
l1_ratio	Elastic Net mixture: 0 = Ridge-like, 1 = Lasso-like.

Strengths

- Reduces variance.
- Lasso can produce sparse coefficients.
- Useful with correlated/high-dimensional features.

Limitations

- Requires tuning.
- Lasso can behave unpredictably with strongly correlated predictors.
- Scaling matters.

Practical implementation

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet

ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.1)
enet = ElasticNet(alpha=0.1, l1_ratio=0.5)
```

16. Logistic Regression

Problem solved: Binary or multiclass classification.

Intuition: Model class probability using a linear score passed through a sigmoid for binary classification.

Core mathematics

$p(y=1|x) = 1/(1+e^{-z})$, where $z = \beta_0 + \beta^T x$. Train by minimizing log loss, usually with regularization.

Training process

Compute linear scores, probabilities and optimize log-loss/regularized objective. A threshold converts probabilities to class predictions.

Important hyperparameters

Hyperparameter	What it controls
C	Inverse regularization strength.
penalty	Regularization type.
solver	Optimization algorithm.

Strengths

- Fast and interpretable.
- Strong baseline.
- Outputs probabilities.

Limitations

- Linear decision boundary in feature space.
- Sensitive to irrelevant features and scaling.
- Needs appropriate encoding.

Practical implementation

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
pred = model.predict(X_test)
proba = model.predict_proba(X_test)[:, 1]
```

17. k-Nearest Neighbors

Problem solved: Classification or regression based on nearby examples.

Intuition: Look at the k closest training points and vote/average their outcomes.

Core mathematics

Common distance: Euclidean $d(x,z)=\sqrt{\sum(x_j-z_j)^2}$. Classification uses majority vote; regression uses neighbor averaging.

Training process

Store training examples. At prediction time, calculate distances to training points, select k nearest and aggregate.

Important hyperparameters

Hyperparameter	What it controls
n_neighbors	Number of neighbors k.
weights	Uniform or distance-based voting.
metric	Distance function.

Strengths

- Simple conceptually.
- Can model nonlinear boundaries.
- No explicit parametric training.

Limitations

- Prediction can be expensive.
- Sensitive to scale.
- Suffers in high dimensions.

Practical implementation

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
```

18. Naive Bayes

Problem solved: Fast probabilistic classification, especially useful for text.

Intuition: Use Bayes theorem and assume features are conditionally independent given the class.

Core mathematics

$P(y|x) \propto P(y) \prod_j P(x_j|y)$. The 'naive' independence assumption simplifies estimation.

Training process

Estimate class priors and feature likelihoods from training data, then compute posterior scores for new examples.

Important hyperparameters

Hyperparameter	What it controls
alpha	Smoothing strength in Multinomial/related variants.
var_smoothing	Variance smoothing in GaussianNB.

Strengths

- Very fast.
- Works well on some high-dimensional sparse problems.
- Needs relatively little training data.

Limitations

- Independence assumption can be unrealistic.
- Probability estimates may be poorly calibrated in some settings.

Practical implementation

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

19. Decision Trees

Problem solved: Classification and regression with nonlinear rules.

Intuition: Recursively split the feature space into regions that become increasingly pure or reduce prediction error.

Core mathematics

For classification, common impurity measures include $Gini = 1 - \sum p_k^2$ and $entropy = -\sum p_k \log_2(p_k)$. Choose splits that maximize impurity reduction.

Training process

Start at the root, evaluate candidate splits, choose the best split, recurse until stopping conditions are met.

Important hyperparameters

Hyperparameter	What it controls
max_depth	Maximum tree depth.
min_samples_split	Minimum samples needed to split.
min_samples_leaf	Minimum samples in a leaf.
criterion	Split quality measure.

Strengths

- Interpretable rules.
- Handles nonlinearities and mixed feature scales.
- No scaling required.

Limitations

- Can overfit badly.
- Small data changes can change the tree.
- Single trees may have high variance.

Practical implementation

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)
model.fit(X_train, y_train)
```

20. Random Forest

Problem solved: Robust classification/regression using many decision trees.

Intuition: Train many trees on bootstrapped samples and random feature subsets; aggregate their predictions.

Core mathematics

Bootstrap aggregation reduces variance. Classification commonly uses majority vote; regression uses averaging.

Training process

Sample training rows with replacement, grow diverse trees, then aggregate their predictions.

Important hyperparameters

Hyperparameter	What it controls
n_estimators	Number of trees.
max_depth	Maximum depth.
max_features	Features considered per split.
min_samples_leaf	Minimum leaf size.

Strengths

- Strong general-purpose baseline.
- Usually less overfit than one tree.
- Handles nonlinear relationships.

Limitations

- Less interpretable than one tree.
- Can be memory intensive.
- Large forests may be slower.

Practical implementation

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    class_weight="balanced"
)
model.fit(X_train, y_train)
```

21. Gradient Boosting

Problem solved: High-performance tabular classification/regression.

Intuition: Build weak learners sequentially; each new learner focuses on correcting errors of the current ensemble.

Core mathematics

At each stage, a learner approximates the negative gradient of the loss with respect to current predictions. The ensemble updates by adding a scaled learner.

Training process

Initialize predictions, compute residual/gradient signal, fit a weak tree, add it with a learning rate, repeat.

Important hyperparameters

Hyperparameter	What it controls
n_estimators	Number of boosting stages.
learning_rate	Contribution of each tree.
max_depth	Complexity of each tree.
subsample	Fraction of samples used per stage.

Strengths

- Excellent tabular performance.
- Flexible nonlinear modeling.
- Can optimize many losses.

Limitations

- Sensitive to tuning.
- Sequential training can be slower.
- Can overfit with too much complexity.

Practical implementation

```
from sklearn.ensemble import GradientBoostingClassifier

model = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=2,
    random_state=42
)
model.fit(X_train, y_train)
```

22. XGBoost / LightGBM / CatBoost

Problem solved: Highly optimized gradient-boosted decision trees for structured/tabular data.

Intuition: All are gradient-boosting families with engineering and algorithmic improvements; exact behavior and strengths differ.

Core mathematics

Tree ensembles optimize a differentiable objective. Modern implementations use regularization, efficient split finding and other techniques.

Training process

Sequentially add trees that reduce the current objective while controlling complexity.

Important hyperparameters

Hyperparameter	What it controls
learning_rate	Step size for boosting.
n_estimators	Number of boosting rounds.
max_depth / num_leaves	Tree complexity.
subsample / feature sampling	Randomization and regularization.
regularization	Controls model complexity.

Strengths

- Often state-of-the-art on tabular problems.
- Handles nonlinear interactions.
- Rich tuning options.

Limitations

- Many hyperparameters.
- Easy to overfit through excessive tuning.
- Interpretation requires care.

Practical implementation

```
# Example XGBoost
from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42
)
model.fit(X_train, y_train)
```

23. Support Vector Machines

Problem solved: Classification and regression, especially useful for medium-sized feature spaces.

Intuition: Find a separating hyperplane with maximum margin. Kernels can create nonlinear decision boundaries.

Core mathematics

Linear SVM maximizes margin while penalizing violations. Soft-margin objective balances margin and classification errors. Kernel methods replace dot products with kernel functions.

Training process

Find a hyperplane or kernel decision function that maximizes margin while controlling violations.

Important hyperparameters

Hyperparameter	What it controls
C	Penalty for margin violations.
kernel	Linear, polynomial, RBF and others.
gamma	Influence of one training example for RBF/poly kernels.
degree	Polynomial kernel degree.

Strengths

- Powerful nonlinear classification.
- Effective in high-dimensional spaces.
- Good with well-scaled features.

Limitations

- Scaling is important.
- Kernel SVM can become expensive on large datasets.
- Probabilities are not native in the same way as logistic regression.

Practical implementation

```
from sklearn.svm import SVC

model = SVC(
    kernel="rbf",
    C=1.0,
    gamma="scale",
    probability=True
)
model.fit(X_train, y_train)
```

24. Neural Networks and Deep Learning Foundations

A neural network is a composition of parameterized functions. A basic neuron computes $z = w \cdot x + b$ and applies an activation function.

Activation	Typical use
ReLU	Hidden layers; $f(x)=\max(0,x)$
Sigmoid	Binary output probability
Softmax	Multiclass probability distribution
Tanh	Older hidden-layer activation; centered around zero

Forward propagation: input passes through layers to produce output. **Loss:** measures prediction error.

Backpropagation: applies the chain rule to compute gradients. **Optimizer:** updates weights using gradients.

Common optimizers: SGD, Momentum, RMSProp, Adam. **Regularization:** weight decay/L2, dropout, early stopping, data augmentation and more.

```
# Minimal conceptual Keras example
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Input(shape=(20,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# model.fit(X_train, y_train, validation_split=0.2, epochs=20)
```

25. Clustering

Clustering is unsupervised learning: there is no target label. The goal is to discover groups with some notion of similarity.

K-Means

Choose K centroids, assign each point to its nearest centroid, recompute centroids, and repeat until assignments stabilize or the objective stops improving.

Objective: minimize within-cluster sum of squared distances.

```
from sklearn.cluster import KMeans

model = KMeans(
    n_clusters=3,
    random_state=42,
    n_init="auto"
)
labels = model.fit_predict(X_scaled)
```

Choosing K: elbow method, silhouette score and domain knowledge. K-Means assumes roughly compact clusters under its distance/objective geometry.

Other clustering algorithms: hierarchical clustering, DBSCAN, HDBSCAN (external library), Gaussian mixture models.

26. PCA and Dimensionality Reduction

Principal Component Analysis transforms correlated numeric variables into orthogonal components ordered by explained variance.

Conceptually: center/scale data when appropriate, compute covariance structure, find eigenvectors/principal directions, project observations into a lower-dimensional space.

Uses: visualization, compression, noise reduction, feature decorrelation and preprocessing in selected workflows.

Caution: PCA components are combinations of features and can be difficult to interpret. Fit PCA only on training data when it is used in a predictive pipeline.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

pca_model = make_pipeline(
    StandardScaler(),
    PCA(n_components=2)
)

X_2d = pca_model.fit_transform(X_train)
```

27. Anomaly Detection

Anomaly detection attempts to identify observations that do not resemble the majority of the data.

Method	Idea
Z-score	Flag observations far from a mean under suitable assumptions
IQR rule	Flag observations outside 1.5×IQR fences
Isolation Forest	Randomly isolate observations; anomalies tend to be isolated quickly
One-Class SVM	Learn a boundary around normal observations
Local Outlier Factor	Compare local density to neighboring points

```
from sklearn.ensemble import IsolationForest
```

```
detector = IsolationForest(  
    contamination="auto",  
    random_state=42  
)  
labels = detector.fit_predict(X_numeric)
```

28. Association Rule Learning

Association rules are commonly used in market-basket analysis to discover items that occur together.

For rule $A \rightarrow B$: **support** measures how often A and B occur together; **confidence** measures $P(B|A)$; **lift** compares confidence with the baseline frequency of B.

Lift > 1 can indicate positive association, but association is not causation.

29. Time Series Foundations

Time-series data is indexed by time. The order of observations matters.

Components: trend, seasonality, cyclic behavior, irregular/noise.

Important rules: parse timestamps correctly, sort chronologically, avoid future leakage, create lag/rolling features using only information available at prediction time.

```
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values("date")
```

```
df["lag_1"] = df["sales"].shift(1)
df["rolling_7"] = df["sales"].shift(1).rolling(7).mean()
```

Traditional models: moving averages, exponential smoothing, ARIMA/SARIMA. **ML approaches:** tree boosting with lag features. **Deep learning:** sequence models when data volume and problem structure justify them.

30. Imbalanced Classification

An imbalanced target has one class much more frequent than another. Accuracy can become misleading.

Example: 99% negatives and 1% positives. A model predicting negative every time gets 99% accuracy but zero recall for the positive class.

Strategies: class weights, threshold tuning, oversampling, undersampling, SMOTE, appropriate metrics and careful cross-validation.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    class_weight="balanced",
    max_iter=1000
)
```

Resampling must happen inside the training folds, not before the split. Otherwise information from validation/test data can leak into training.

31. Model Evaluation

Regression metrics

Metric	Formula / meaning
MAE	Mean absolute error; easy to interpret in target units
MSE	Mean squared error; penalizes large errors more
RMSE	$\sqrt{\text{MSE}}$; same units as target
R ²	Fraction of variance explained relative to a mean baseline under the usual definition
MAPE	Mean absolute percentage error; problematic when actual values are zero/near zero

Classification confusion matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Accuracy = $(\text{TP} + \text{TN}) / \text{total}$

Precision = $\text{TP} / (\text{TP} + \text{FP})$

Recall = $\text{TP} / (\text{TP} + \text{FN})$

F1 = $2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$

Specificity = $\text{TN} / (\text{TN} + \text{FP})$

ROC-AUC: summarizes ranking performance across classification thresholds using true-positive rate versus false-positive rate. **PR-AUC:** is often more informative when the positive class is rare.

Calibration: if a model predicts 0.8 probability for many cases, roughly 80% of those cases should be positive for good calibration.

32. Hyperparameter Tuning

Hyperparameters are settings that control the learning procedure/model structure and are not learned as ordinary model parameters from the training objective.

Examples: K in KNN, max_depth in trees, C and gamma in SVM, learning_rate and n_estimators in boosting, alpha in regularized linear models, hidden units/learning rate/batch size/epochs in neural networks.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "max_depth": [3, 5, 8],
    "min_samples_leaf": [1, 2, 5]
}

search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="f1"
)

search.fit(X_train, y_train)

print(search.best_params_)
print(search.best_score_)
```

Grid search: evaluates all specified combinations. **Random search:** samples combinations and can be more efficient in large spaces. **Bayesian optimization:** uses previous evaluations to choose promising configurations.

33. Cross-Validation

K-fold cross-validation divides training data into K folds. Each fold acts as validation once while the other K-1 folds train the model.

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

scores = cross_val_score(
    model,
    X_train,
    y_train,
    cv=cv,
    scoring="f1"
)

print(scores)
print(scores.mean())
```

Use stratified folds for classification when preserving class proportions is important. For grouped observations, use group-aware splitting. For time series, use time-aware splits.

34. Explainability

Model explainability helps answer: Why did the model make this prediction? Which features influence predictions? Does the model rely on plausible signals?

Global methods: permutation importance, model coefficients, tree-based importance, partial dependence.

Local methods: SHAP-style additive explanations and LIME-style local surrogate explanations.

Caution: feature importance is not automatically causal importance. Correlated features can distribute importance in unintuitive ways.

```
from sklearn.inspection import permutation_importance

result = permutation_importance(
    model,
    X_test,
    y_test,
    n_repeats=10,
    random_state=42
)

print(result.importances_mean)
```

35. Complete ML Project Workflow

Example problem: predict whether a student will be placed.

```
# 1. Load
df = pd.read_csv("students.csv")

# 2. Define target
X = df.drop(columns=["placement", "student_id"])
y = df["placement"]

# 3. Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 4. Build preprocessing
# numeric -> impute + scale
# categorical -> impute + one-hot

# 5. Attach model
# pipeline = Pipeline([...])

# 6. Fit only on training data
# pipeline.fit(X_train, y_train)

# 7. Predict
# pred = pipeline.predict(X_test)

# 8. Evaluate
# classification_report(y_test, pred)

# 9. Tune using CV
# 10. Lock final model
# 11. Save
# 12. Deploy and monitor
```

Project documentation should include: problem statement, data source, target definition, data dictionary, cleaning decisions, EDA findings, feature engineering, split strategy, baseline, models tried, metrics, tuning, error analysis, limitations, fairness/safety considerations and deployment plan.

36. Model Saving and Deployment

A trained model is only useful if the same preprocessing and feature logic can be applied to future data.

```
import joblib

# Save complete pipeline
joblib.dump(pipeline, "placement_model.joblib")

# Load later
loaded = joblib.load("placement_model.joblib")
prediction = loaded.predict(new_data)
```

Typical deployment architecture: frontend → REST API → preprocessing/model pipeline → prediction → response.
Common Python API frameworks include Flask and FastAPI.

Production concerns: input validation, versioning, logging, latency, monitoring, model drift, data drift, security, reproducibility, rollback and retraining.

37. Common Mistakes and Interview Questions

Common mistakes

- Scaling before splitting the data and thereby leaking test information.
- Imputing missing values using the complete dataset before splitting.
- Selecting features using the target/test set before validation.
- Using accuracy alone on severely imbalanced data.
- Removing all outliers without checking whether they are valid.
- Treating IDs as meaningful predictive features.
- Using random train/test splitting for time-series forecasting.
- Tuning repeatedly against the test set.
- Ignoring a simple baseline model.
- Assuming correlation proves causation.
- Using complex models without enough data or without a reason.
- Not checking what features will actually be available at prediction time.

Interview questions to master

- What is the difference between supervised and unsupervised learning?
- What is overfitting and how do you reduce it?
- What is the difference between a parameter and a hyperparameter?
- Why do we split data into train and test sets?
- Why is cross-validation used?
- When should you use precision vs recall?
- Why can accuracy be misleading?
- What is regularization?
- Difference between L1 and L2 regularization?
- Why does KNN need scaling?
- Why do tree-based models usually not require scaling?
- What is bagging vs boosting?
- How does Random Forest reduce variance?
- What is gradient boosting?
- What is data leakage?
- What is one-hot encoding?
- What is feature engineering?
- What is multicollinearity?
- What is ROC-AUC?
- When would you choose a simpler model over XGBoost?

38. Practical Study Plan

Because you want depth rather than a rushed 15-day overview, use the following sequence. Each day should include reading + implementation + a mini exercise.

Phase	Topics	Practical output
Days 1–3	Python, NumPy, Pandas, cleaning, EDA	Clean and analyze a real CSV
Days 4–5	Statistics, probability, visualization	EDA report + statistical analysis
Days 6–7	Preprocessing, feature engineering, leakage, splitting	Reusable preprocessing pipeline
Days 8–10	Regression + classification algorithms	Train and compare multiple models
Days 11–12	Trees, ensembles, boosting, SVM, Naive Bayes, KNN	Model comparison experiment
Days 13–14	Clustering, PCA, anomaly detection, imbalance	Unsupervised + imbalanced-learning notebook
Day 15	Evaluation, tuning, CV, explainability, deployment	End-to-end ML project
After 15	Deep learning, NLP, CV, time series, MLOps	Specialization projects

For every algorithm, use this template:

1. What problem does it solve?
2. Intuition
3. Mathematical objective
4. Assumptions
5. How training works
6. How prediction works
7. Important hyperparameters
8. Advantages
9. Limitations
10. When to use it
11. When not to use it
12. Implement from scratch when practical
13. Implement with scikit-learn
14. Evaluate on a real dataset
15. Tune hyperparameters
16. Analyze errors

Recommended project progression: house-price regression → student-placement classification → customer churn → fraud/imbalanced classification → clustering customer segments → time-series forecasting → one end-to-end deployed project.

Quick Reference — Essential scikit-learn Imports

```
# Splitting / CV
from sklearn.model_selection import train_test_split, KFold, StratifiedKFold
from sklearn.model_selection import cross_val_score, GridSearchCV, RandomizedSearchCV

# Preprocessing
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.preprocessing import OneHotEncoder, PolynomialFeatures
from sklearn.impute import SimpleImputer

# Pipeline
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# Regression
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet

# Classification
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

# Unsupervised
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Metrics
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score,
    average_precision_score
)
```

Final mental model: Data Science is not just coding models. The hardest and most important work is often defining the target correctly, understanding the data, preventing leakage, choosing meaningful evaluation criteria, engineering useful features, validating honestly and communicating what the model can and cannot claim.

Study rule: For each chapter, execute the code yourself and change the dataset/parameters. A model you have trained, evaluated and debugged is worth much more than a model you have only read about.